Trees and Ensembles

Lesson 7 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · reading time about 85 minutes

Contents

Lesson plan . . . 1
1. Why this lesson exists . . . 2
    1.1 The dataset, and the shape a line cannot cover . . . 2
2. Decision trees: splitting on Gini impurity . . . 4
    2.1 The whole algorithm . . . 4
    2.2 Impurity, and the gain a split buys . . . 4
3. Depth is the bias-variance dial . . . 6
4. Reading a tree, and why it doesn't stay still . . . 8
5. Bagging: averaging away the instability . . . 9
    5.1 How much data one bootstrap sample actually sees . . . 9
    5.2 Why averaging reduces variance, and what it cannot fix . . . 10
6. Random forests: decorrelating the trees . . . 12
7. Feature importance, and its limits . . . 13
8. Gradient boosting: correcting mistakes in sequence . . . 15
9. Boosting for classification: the same idea, a different gradient . . . 16
10. Overfitting in boosting: two knobs that trade off . . . 17
11. Choosing among trees, forests and boosting . . . 19
Summary . . . 20
    Homework . . . 20
    Notation used in this lesson . . . 20
Further reading . . . 21

Lesson plan

Time	Minutes	Segment	Material
0:00–0:10	10	Exercise 6 discussed; the choice this lesson offers	Slides 2–5
0:10–0:32	22	Decision trees: splitting on Gini impurity	Slides 6–18
0:32–0:52	20	Depth is the bias-variance dial	Slides 19–24

Time	Minutes	Segment	Material
0:52–1:12	20	**Notebook 01** — decision trees from scratch	Slide 25
1:12–1:24	12	**Break**	Slide 26
1:24–1:44	20	Bagging and random forests	Slides 27–37
1:44–2:02	18	**Notebook 02** — bagging and random forests	Slide 38
2:02–2:24	22	Gradient boosting	Slides 39–48
2:24–2:42	18	**Notebook 03** — gradient boosting	Slide 49
2:42–3:00	18	The full leaderboard; homework	Slides 50–55
	180	**Total**	**55 slides, 3 notebooks**

---

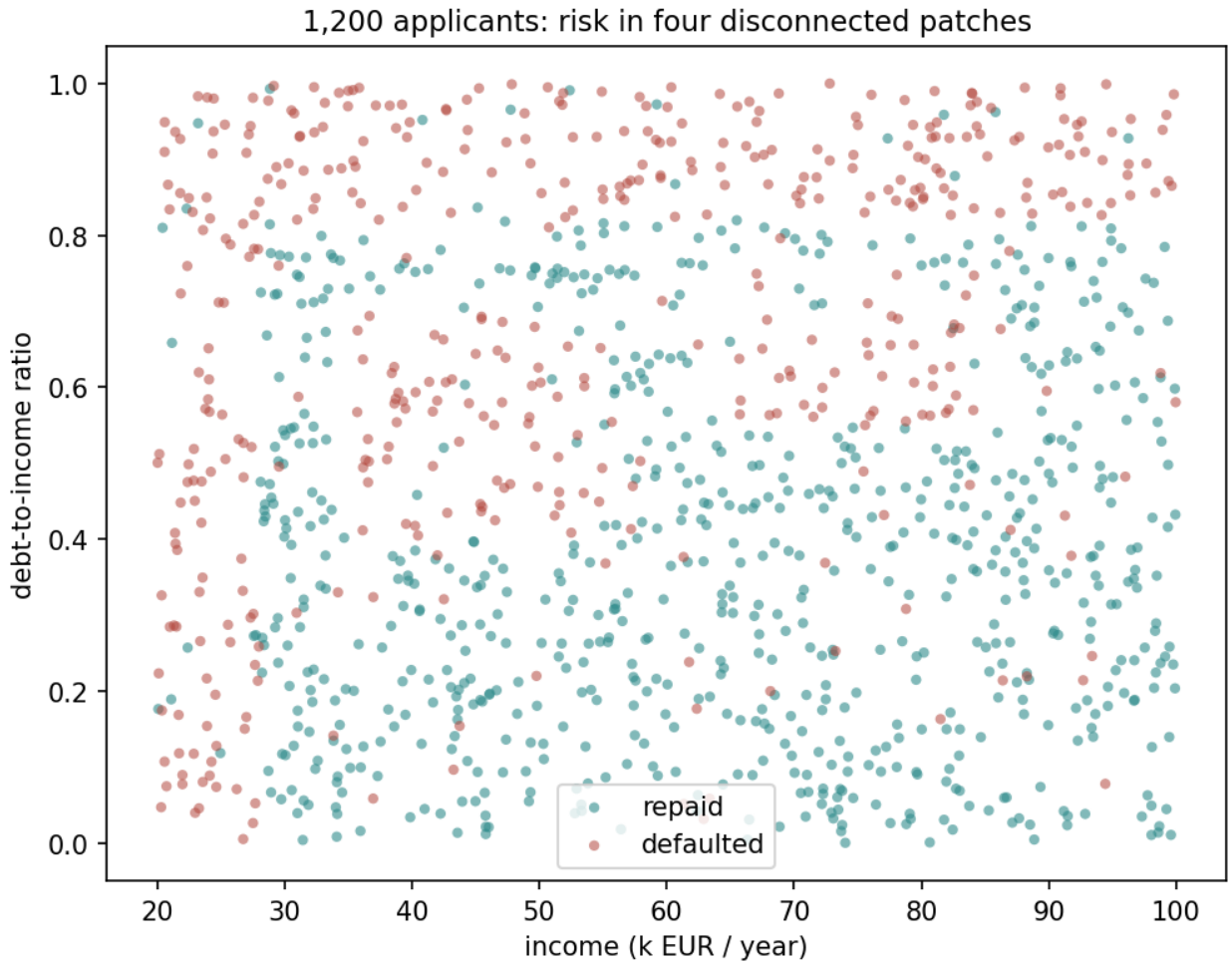
1. Why this lesson exists

Lessons 3 and 4 fit a straight line and made it honest. Lesson 6 offered three ways of bending, remembering or assuming your way around a boundary a line cannot draw. This lesson introduces a fourth family, built on a completely different primitive — a threshold, not a distance or an inner product — and then spends most of its time on the question that actually decides how well trees work in practice: what happens when you build more than one.

A single decision tree is one of the few methods in this course a person without a mathematics background can read end to end. It is also, on its own, a mediocre model: accurate enough to be tempting, unstable enough to be untrustworthy. **Ensembles of trees — bagging, random forests, gradient boosting — are the method most competition leaderboards on tabular data are still won with**, precisely because they turn that instability into their raw material rather than trying to eliminate it.

1.1 The dataset, and the shape a line cannot cover

1,200 loan applicants, two figures each at the point of application: income in thousands of euros a year, and debt-to-income ratio. Whether the loan later defaulted is not a straight function of either. A bank's underwriting logic is closer to a checklist than a formula: an income floor below which almost nobody is approved regardless of anything else, a debt ceiling above which almost nobody is, and two "stressed" combinations in between — income and debt levels that are each unremarkable alone but risky together.



1,200 applicants. The risky region is not one shape but four: a strip of low income, a strip of high debt, and two disconnected rectangles where income and debt are each ordinary but the combination is not. No single straight line separates repaid from defaulted, because the class you would have to isolate is not even one connected region.

7% of labels are flipped after the rule is applied — the noise floor this lesson works against, and the reason no model below should be read as failing when it falls short of 1.000. 38.7% of applicants defaulted (464 of 1,200), so the majority baseline is 0.613.

Model	Cross-validated accuracy
Always predict the majority class	0.613
Logistic regression	**0.748 ± 0.030**

Unlike lesson 6's pump data — where logistic regression matched the baseline exactly and had learned nothing — here it clears the baseline by a wide margin. Two of the four risky regions, the low-income strip and the high-debt strip, are monotonic in one feature — the risk rises steadily as income falls or as debt rises — and a linear boundary captures monotonic effects reasonably well. What it cannot do is wall off the two *interior* islands without cutting into healthy territory everywhere else, and closing that gap is what the rest of this lesson is about.

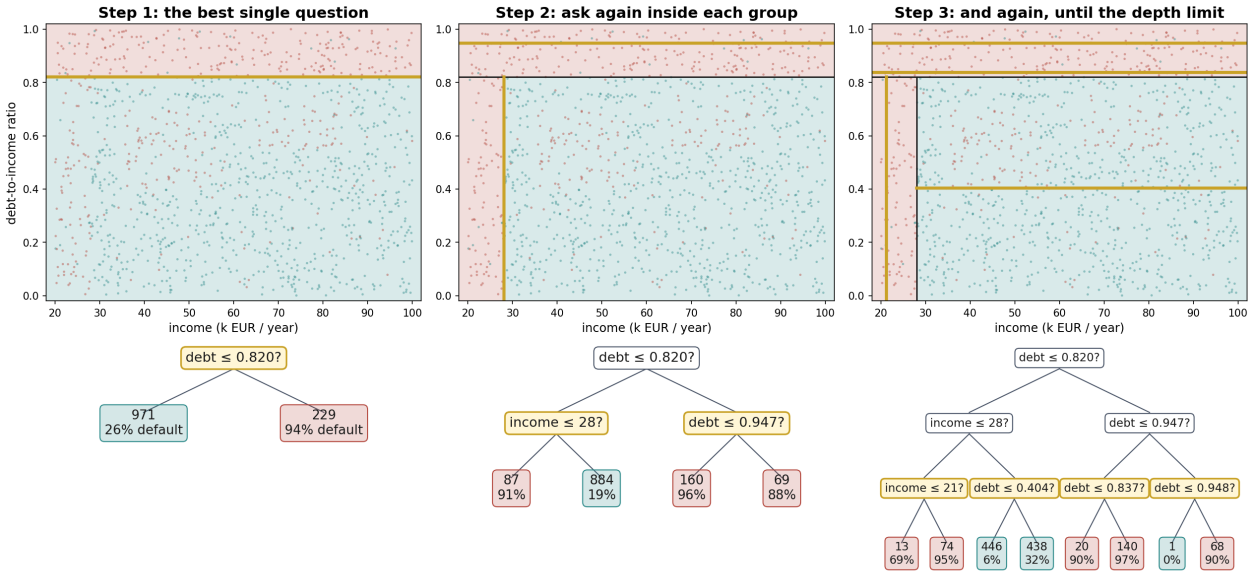
2. Decision trees: splitting on Gini impurity

2.1 The whole algorithm

> Find the single (feature, threshold) split that makes the two resulting groups as pure as possible. Apply it. Repeat on each group, independently, until a stopping rule is met.

Nothing is estimated in the sense lessons 3 and 4 used the word — no coefficient, no gradient with respect to a parameter vector. A tree is a sequence of yes/no questions, chosen greedily, and "greedily" matters: at every step the tree takes whichever split helps most *right now*, with no mechanism for looking ahead to a split that would help more two levels down.

The recursion is easiest to see run. Three rounds of it on the loan data:



Each column is one more round of "find the best question, apply it, repeat inside each group". On top, the cuts in the plane of the data, the ones added in that round in gold; underneath, the tree asking the same questions, each leaf with its applicants and default rate. Step 2 asks a different question in each group, because each is searched on its own. Step 3 shows greed: one cut, at 0.948 — a thousandth above the one at 0.947, so the two gold lines overlap — sets aside a single applicant from 68, another splits 160 into 90% and 97% default and changes no prediction, and neither interior island has been found yet.

2.2 Impurity, and the gain a split buys

"As pure as possible" needs a number. For classification, the standard choice is the **Gini impurity** of a group of m_g examples drawn from C classes:

$$G = 1 - \sum_{c=1}^{C} p_c^2$$

where p_c is the fraction of the group belonging to class c. For two classes this simplifies to $2p(1-p)$: expand $(1-p)^2$ as $1-2p+p^2$, and the two p^2 terms and the two 1s cancel, leaving $G = 1-p^2-(1-2p+p^2) = 2p-2p^2$. It is 0 when the group is pure ($p \in \{0,1\}$) and maximal, 0.5, at $p = 0.5$.

What that number is, exactly. G is not a probability of misclassification. It is the probability that two examples drawn at random *with replacement* from the group, and labelled according to the group's own class frequencies, would **disagree** — and the formula is that statement written out. Draw twice: the two draws agree on class c with probability p_c^2, so they agree at all with probability $\sum_c p_c^2$, and they disagree with the rest:

$$P(\text{disagree}) = 1 - \sum_{c=1}^{C} p_c^2 = G$$

A pure group cannot disagree with itself, which is why $G = 0$ there.

A candidate split divides a parent group of m_P examples into a left child of m_L examples and a right child of $m_R = m_P - m_L$. Its value is the reduction in impurity, weighted by how the group is divided:

$$\Delta G = G(\text{parent}) - \left(\frac{m_L}{m_P} G(\text{left}) + \frac{m_R}{m_P} G(\text{right}) \right)$$

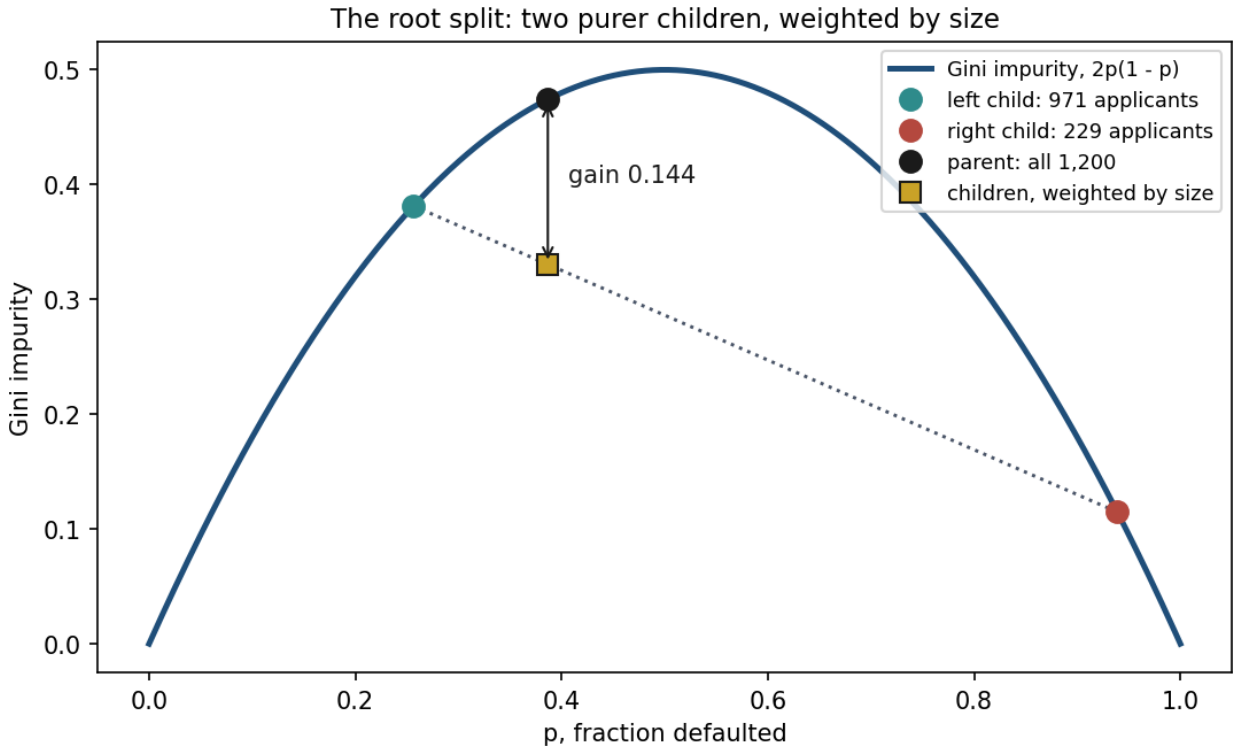
For a continuous feature, the tree scans every feature and, within each, every midpoint between two adjacent sorted values as a candidate threshold, and keeps whichever (feature, threshold) pair maximises ΔG. It then repeats the entire search independently inside each child. This is what makes the algorithm greedy: ΔG is evaluated one split at a time, never for a pair of splits jointly, so a feature that only helps in combination with a second split further down can be — and in this dataset's two interior islands, is — invisible to a shallow tree.

Worked example. On the full 1,200 applicants, $p = 0.387$ defaulted, so $G(\text{parent}) = 2 \times 0.387 \times 0.613 = 0.474$. The best root split found by exhaustive search is `debt_ratio <= 0.82` — recognisably the debt ceiling the data was built with — and the from-scratch implementation below confirms this is exactly the split scikit-learn's `DecisionTreeClassifier` also finds. More than that: at every depth tested the two make the same prediction for every one of the 1,200 applicants. The check runs on the training data on purpose — it tests the code, whether two programs build the same tree from the same rows, not how well the tree will do on applicants it has not seen.

What that split buys, in the formula above. It sends 971 applicants left, of whom $p = 0.256$ defaulted, and 229 right, of whom $p = 0.939$ did:

Group	Applicants	p	$G = 2p(1-p)$
Parent	1,200	0.387	0.4743
Left, `debt_ratio <= 0.82`	971	0.256	0.3814
Right	229	0.939	0.1148

$$\Delta G = 0.4743 - \left(\tfrac{971}{1200} \times 0.3814 + \tfrac{229}{1200} \times 0.1148 \right) = 0.4743 - 0.3305 = 0.1438$$



The Gini curve $2p(1-p)$ with the parent (black) and the two children (teal and rust). The children's size-weighted average (gold) lies on the straight line between them, and the curve bulges above every such line — so any split that separates the classes at all lowers the weighted impurity. The gap, 0.144, is the gain; the tree takes the split whose gap is largest.

> **Try this:** using `Notebooks/loan_data.py`, compute G for the group of applicants with `income_k < 28` (the income-floor rule alone) and compare it with G for the full dataset. It should be substantially lower — that subgroup is disproportionately defaults — which is exactly why the tree is willing to spend a split isolating it.

---

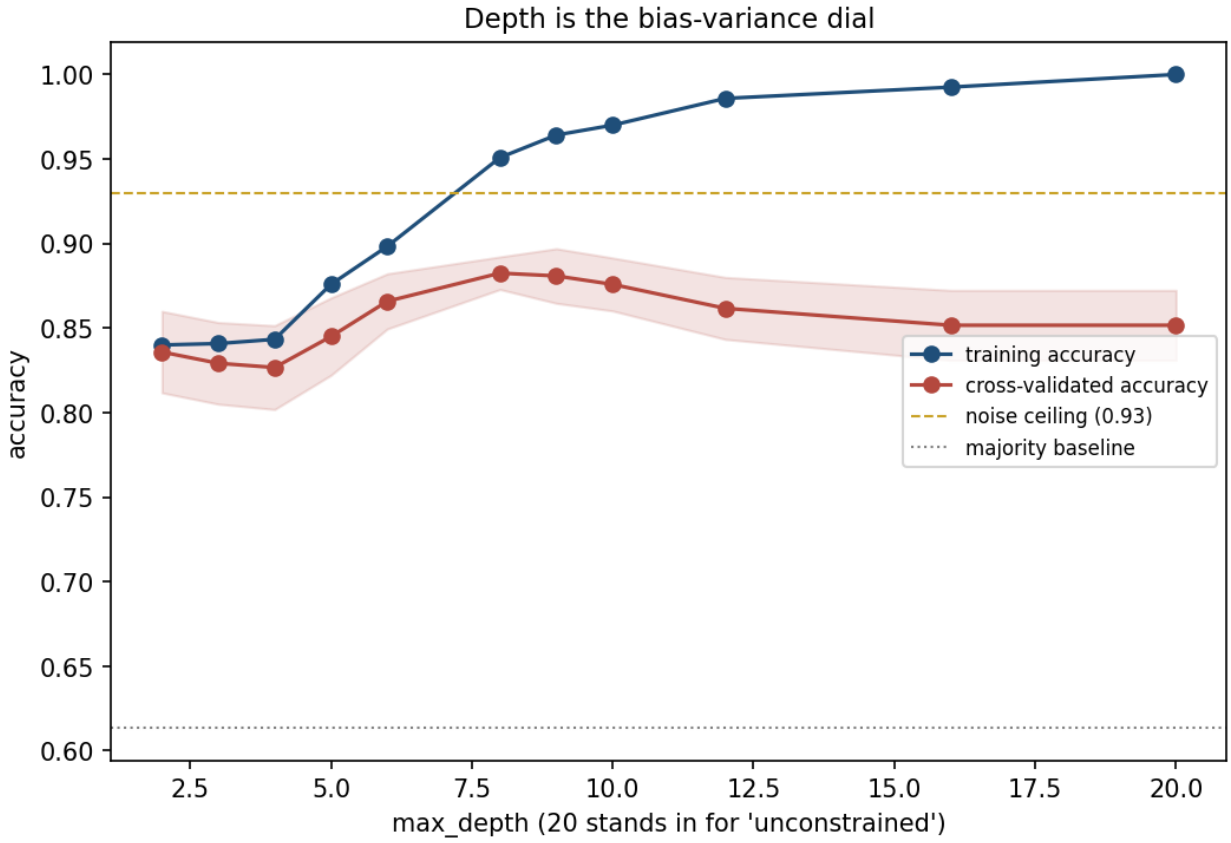
3. Depth is the bias-variance dial

Lesson 6 turned k in k-nearest neighbours and watched bias trade directly against variance. A tree's equivalent dial is **depth**: how many questions it may ask before a leaf must stop and predict a class by majority vote.

Shallow tree. Few questions, so only the coarsest structure can be described. High bias, low variance.

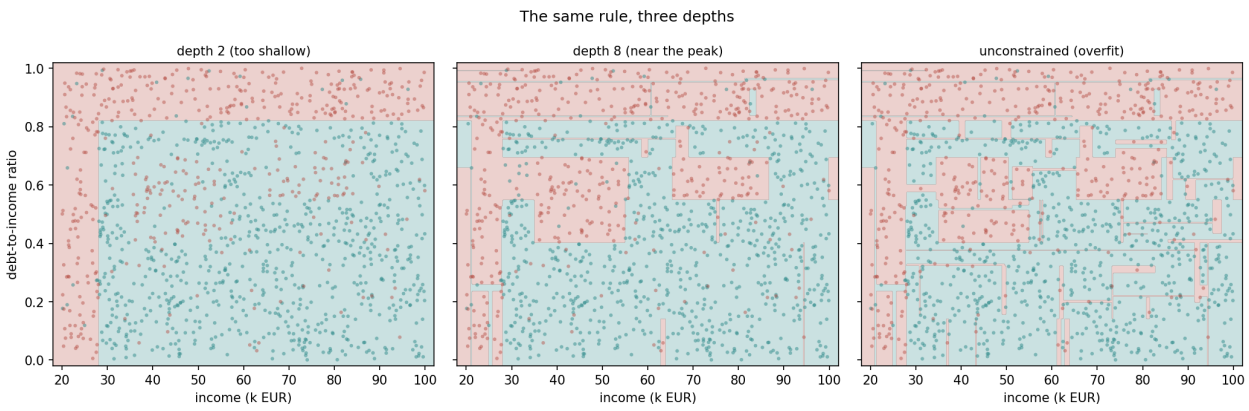
Deep tree. Enough questions to isolate almost any subset of the training data, down to single points. At full depth, training accuracy is **exactly 1.000**, even on a dataset with no structure at all: keep splitting and every leaf is eventually pure, at worst one example, which it then "predicts" perfectly. The one exception is two examples with identical feature values and different labels, which no split can separate; with two continuous features drawn at random, as here, that does not occur. That number demonstrates the model class is flexible enough to memorise, not that anything has been learned — the same reading lesson 6 gave $k = 1$'s training accuracy of 1.000.

`max_depth`	Training accuracy	Cross-validated accuracy
2	0.840	0.836 ± 0.024
4	0.843	0.827 ± 0.025
6	0.898	0.866 ± 0.016
8	0.951	**0.882 ± 0.010**
9	0.964	0.881 ± 0.016
12	0.986	0.862 ± 0.018
16	0.993	0.852 ± 0.021
unconstrained	**1.000**	0.852 ± 0.021



Training accuracy climbs monotonically towards 1.000. Cross-validated accuracy rises, peaks at depth 8, and then falls back — even though the model keeps getting more flexible. Past the peak, every extra split is spent fitting some of the 7% of labels this dataset deliberately flips, and a flipped label has no pattern to learn; a tree that classifies it "correctly" on the training set has memorised it, not generalised.

Read the two columns against each other, never in isolation. The peak sits at depth 8 — deep enough to isolate both stressed islands, shallow enough to leave the flipped labels as errors rather than as leaves of their own. The unconstrained tree **loses 3 points of cross-validated accuracy relative to the peak** while gaining nothing but a training score nobody should have trusted in the first place.



The same rule at three depths. At depth 2 the tree affords only the income floor and the debt ceiling — the two biggest, easiest-to-find regions — and neither interior island is visible at all. At depth 8 both islands appear as clean rectangles, close to the rule that generated the data. At full depth the boundary has grown a fringe of tiny rectangles chasing individual flipped labels: not a smoother version of the depth-8 boundary, a noisier one.

The predictable mistake, and why the instinct is sound. It is entirely reasonable to expect a more flexible model to do at least as well as a less flexible one on held-out data — more options can only help, surely. The instinct is sound for *bias*: a deeper tree can represent anything a shallower one can, plus more. It is wrong for the quantity that is actually measured, because flexibility increases variance at the same time, and past some point the added variance costs more than the reduced bias saves. The course has shown this curve bend before — lesson 3's test error turned back up as the polynomial degree grew, lesson 6's as k shrank towards 1 — and it will not be the last time.

---

4. Reading a tree, and why it doesn't stay still

`max_depth` grows a tree **breadth-first**: every leaf at level d is split before any leaf reaches level $d+1$, whether or not that particular split is worth having. `max_leaf_nodes` grows it **best-first** instead, always expanding whichever current leaf offers the largest ΔG, so a small budget of splits goes to the ones that matter most. For a tree meant to be read rather than merely scored, that is the better knob.

With `max_leaf_nodes = 9` the tree reaches 90.8% training accuracy in nine leaves, and every threshold is recognisable from the generating rule: 28 for the income floor, 0.82 for the debt ceiling, 34.92 and 58.00 tracing the outline of the first stressed island. That printout is the entire model — a sequence of if/else statements a loan officer could apply by hand — and it is the thing k-nearest neighbours and support vector machines cannot offer at all. For a decision that has to be justified to the person it affects, that can matter as much as accuracy; this lesson's companion reading takes the point further.

The cost is **instability**. An early split near the root reroutes everything beneath it, so a small change in the training data can change the tree's shape. Two trees at `max_depth = 6`, each fit to its own random 65% of the 1,200 applicants (drawn without replacement), find the debt ceiling within 0.001 of each other — the evidence for it is overwhelming, so it barely moves — but grow 31 and 33 leaves respectively and **disagree on 11.7% of predictions** on the full dataset. The strongest split is stable because the signal behind it is unambiguous; the weaker splits, each supported by far fewer

points, are not, and they are most of what determines a tree's final shape.

A single tree's instability looks like a flaw. It is the raw material sections 5–8 turn into a strength.

---

5. Bagging: averaging away the instability

If a tree's mistakes are somewhat random — different across resamples of the same data, as section 4 just measured — then many trees, each grown on its own resample and then averaged, should have mistakes that partly cancel. That is **bagging** (**b**ootstrap **agg**regat**ing**):

> Draw a bootstrap sample: m rows chosen **with replacement** from the m training rows, so some appear more than once and some not at all. Fit a tree to it. Repeat `n_estimators` times. To predict, ask every tree and take the majority vote.

Nothing about the tree itself changes — bagging reuses section 2's algorithm unmodified and changes only what surrounds it.

5.1 How much data one bootstrap sample actually sees

Each bootstrap sample is drawn from the same m rows with replacement, so the chance any one specific row is never drawn across m draws is

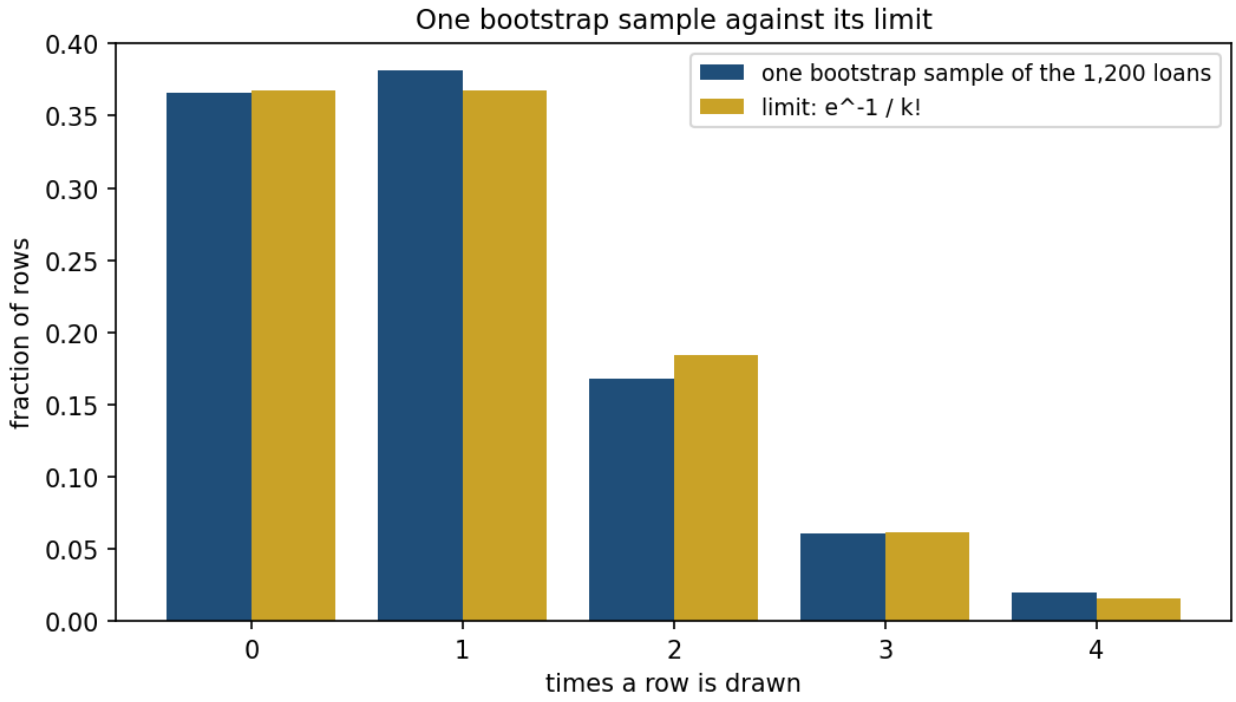
$$\left(1 - \frac{1}{m}\right)^m$$

As $m \to \infty$, this is the standard exponential limit $\lim_{m\to\infty} \left(1 + \frac{x}{m}\right)^m = e^x$ with $x = -1$:

$$\left(1 - \frac{1}{m}\right)^m \xrightarrow{m\to\infty} e^{-1} \approx 0.368$$

so each bootstrap sample contains about $1-e^{-1} \approx 63.2\%$ of the distinct rows and leaves about **36.8%** out entirely — called **out-of-bag** (OOB). At $m = 1{,}200$ the finite-sample value is $(1-1/1200)^{1200} = 0.3677$, already within 0.0002 of the limit, and a single simulated bootstrap draw left out 36.6% of rows — close to both, with the residual gap being exactly the sampling noise a single draw carries.

The same reasoning gives the whole distribution of how often a row is drawn: in the limit, a fraction $e^{-1}/k!$ of rows appears exactly k times — 36.8% never, 36.8% once, 18.4% twice, 6.1% three times. The number of times one row is drawn is binomial, m draws each with probability $1/m$; as m grows with the mean held at $m \cdot 1/m = 1$, the binomial tends to the Poisson distribution with mean 1, whose probabilities are exactly $e^{-1}/k!$.



One bootstrap sample of the 1,200 loans (blue) against the limit $e^{-1}/k!$ (gold). The first bar is the out-of-bag share; the rest are rows the tree sees once or several times. About one row in four is drawn two or more times, and that duplication is what makes each tree different from its neighbour.

Every OOB row is free validation data for the trees that did not see it. Predicting each row by a vote of only those trees, and scoring those predictions, gives the **OOB score**, with no rows set aside. On this dataset a 300-tree random forest — bagging with one extra rule, given in section 6; here it can be read as 300 bagged trees — had an OOB score of **0.9117**, against a 5-fold cross-validated accuracy of **0.9117 ± 0.021** for the same forest.

That the two match to four decimal places is this seed's luck, and it is worth saying so rather than letting the coincidence stand as the demonstration. Repeating both over twelve seeds gives a typical gap of **0.0026** and a worst case of **0.0075**; the exact four-decimal match turns up twice in the twelve. What the theory predicts is the weaker and more useful claim: OOB score *is* cross-validation, with each tree supplying its own held-out fold instead of the dataset being split up front, so the two estimate the same quantity and agree **to within their own noise** — about ±0.003 here, against a cross-validation spread of ±0.021.

The practical consequence is unchanged: with a bagged ensemble you get a trustworthy estimate of generalisation without setting aside a validation set at all. It is simply not an estimate to read to the fourth decimal.

5.2 Why averaging reduces variance, and what it cannot fix

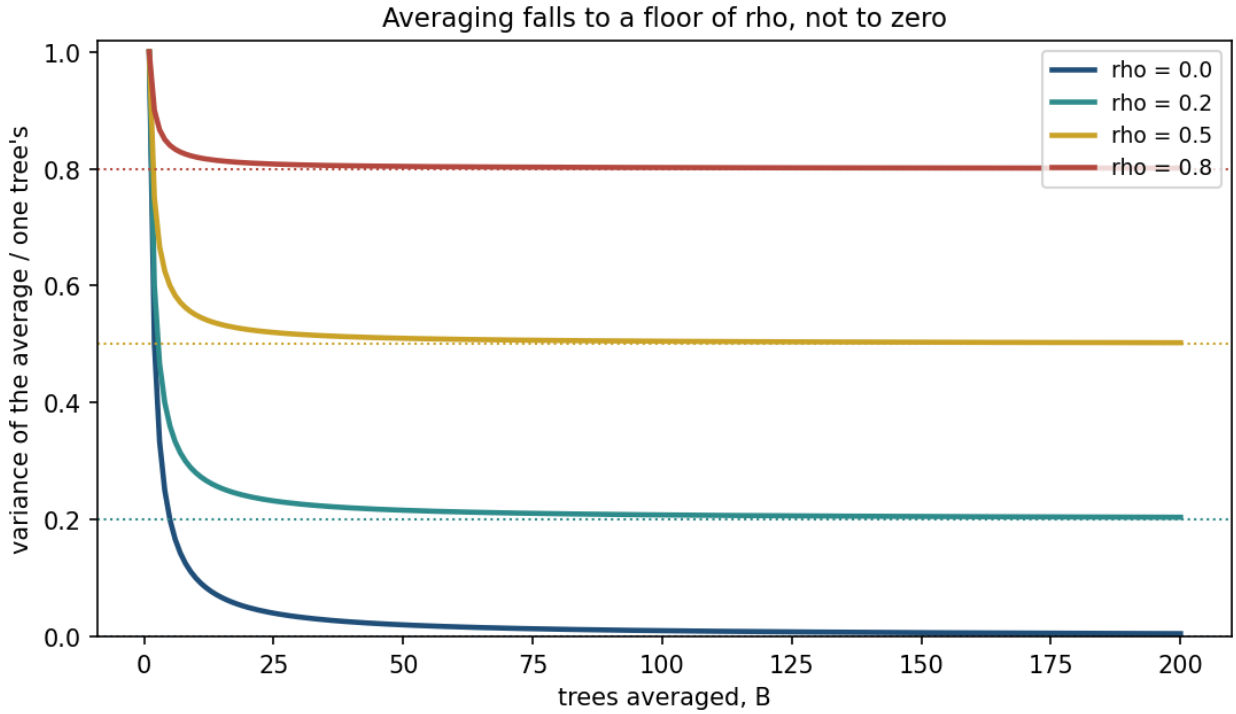
Model each tree's prediction as a random variable with variance σ^2 (over the randomness of which bootstrap sample it happened to see), and let ρ be the average correlation between the predictions of any two trees. For B trees averaged together:

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} f_b\right) = \frac{1}{B^2}\left(\sum_{b=1}^{B}\mathrm{Var}(f_b) + \sum_{b\neq b'}\mathrm{Cov}(f_b, f_{b'})\right)$$

There are B variance terms, each σ^2, and $B(B-1)$ covariance terms, each $\rho\sigma^2$ — the same accounting lesson 5, Section 4.3 did for k correlated fold scores, which is not a coincidence: averaging correlated measurements is one piece of arithmetic, and this is its second appearance in the course:

$$= \frac{1}{B^2}\Big(B\sigma^2 + B(B-1)\,\rho\sigma^2\Big) = \frac{\sigma^2}{B} + \frac{B-1}{B}\,\rho\sigma^2 \xrightarrow{B\to\infty} \rho\sigma^2$$

Two consequences follow directly from the algebra. First, variance falls towards a **floor** of $\rho\sigma^2$, not to zero: adding trees past the point where σ^2/B is small buys almost nothing, because the $\rho\sigma^2$ term does not shrink with B at all. Second, bagging cannot touch **bias** — averaging unbiased-but-noisy trees gives an unbiased average, and averaging trees that share a systematic error preserves that error exactly. Bagging is a variance tool. It does nothing for a tree that is too shallow to represent the rule in the first place.



The variance of an average of B trees, as a share of one tree's, for four values of ρ. Uncorrelated trees average it away; correlated ones stop at the dotted floor however many are added — at $\rho = 0.5$, a hundred trees still keep 0.505 of one tree's variance. Lowering ρ is the only way through the floor, and it is what section 6 does.

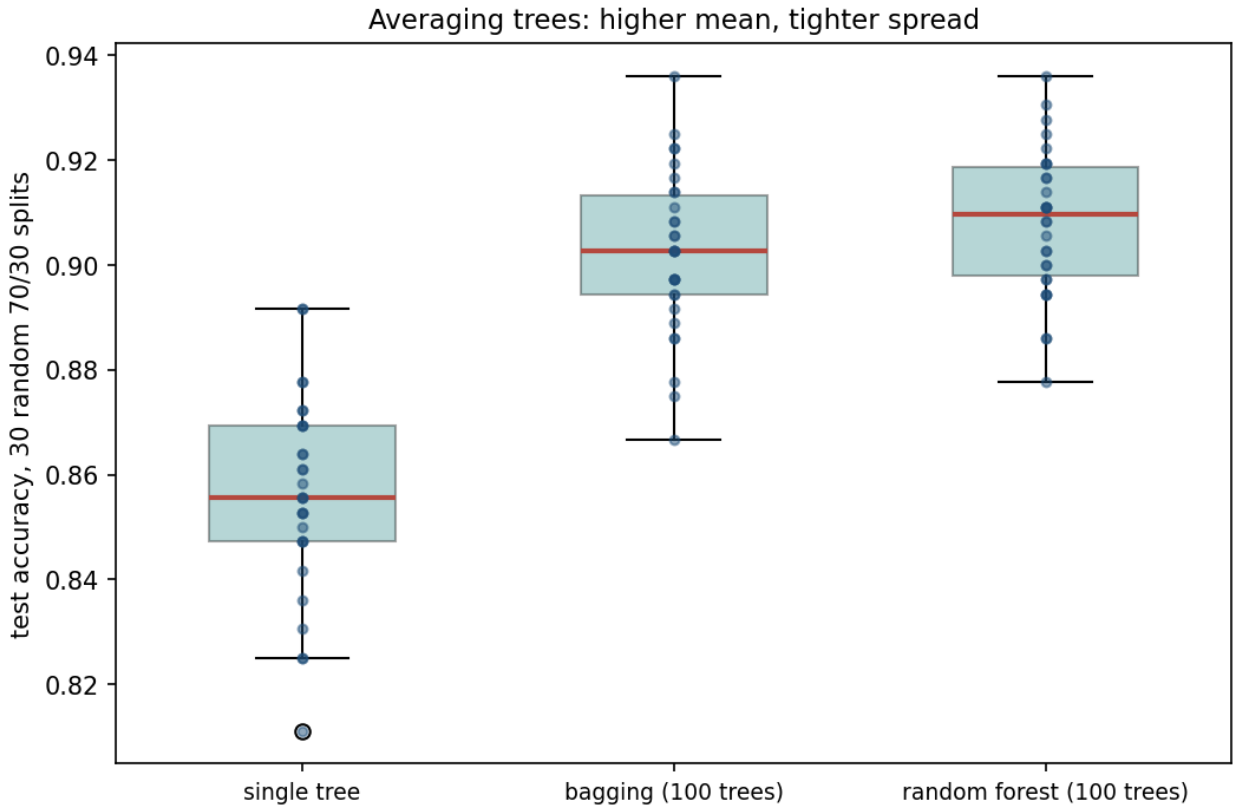
Worked example. Across 30 independent 70/30 train/test splits of the loan data, a single unconstrained tree's test accuracy has mean **0.856** and standard deviation **0.0185**; a 100-tree bagged ensemble has mean **0.902** and standard deviation **0.0153**. Both numbers moved in the predicted direction — the mean rose because bagging also averages out some of a deep tree's overfitting, the spread fell because that is what the formula above says averaging correlated estimators does.

6. Random forests: decorrelating the trees

The variance floor in section 5.2 is $\rho\sigma^2$, so shrinking it further means shrinking ρ — making the trees agree with each other less. Bootstrap resampling alone does this only partly, because bagged trees still consider every feature at every split, so a strong feature pulls most bootstrap trees towards splitting on it first regardless of which rows they drew.

A **random forest** adds one restriction: at each split, only a random subset of `max_features` features — by default $\lfloor\sqrt{n}\rfloor$ for classification — is even offered as a candidate. Different trees are forced to consider different features at the same point in the tree, which lowers ρ and lets the variance floor drop further, without changing the splitting rule itself. It is bagging with the menu of candidate splits shrunk at random, every time a split is made.

Measured on the same 30 splits as section 5.2, a 100-tree random forest reaches mean accuracy **0.908** with standard deviation **0.0137** — both a better mean and a tighter spread than plain bagging's 0.902 and 0.0153, consistent with ρ having fallen further.



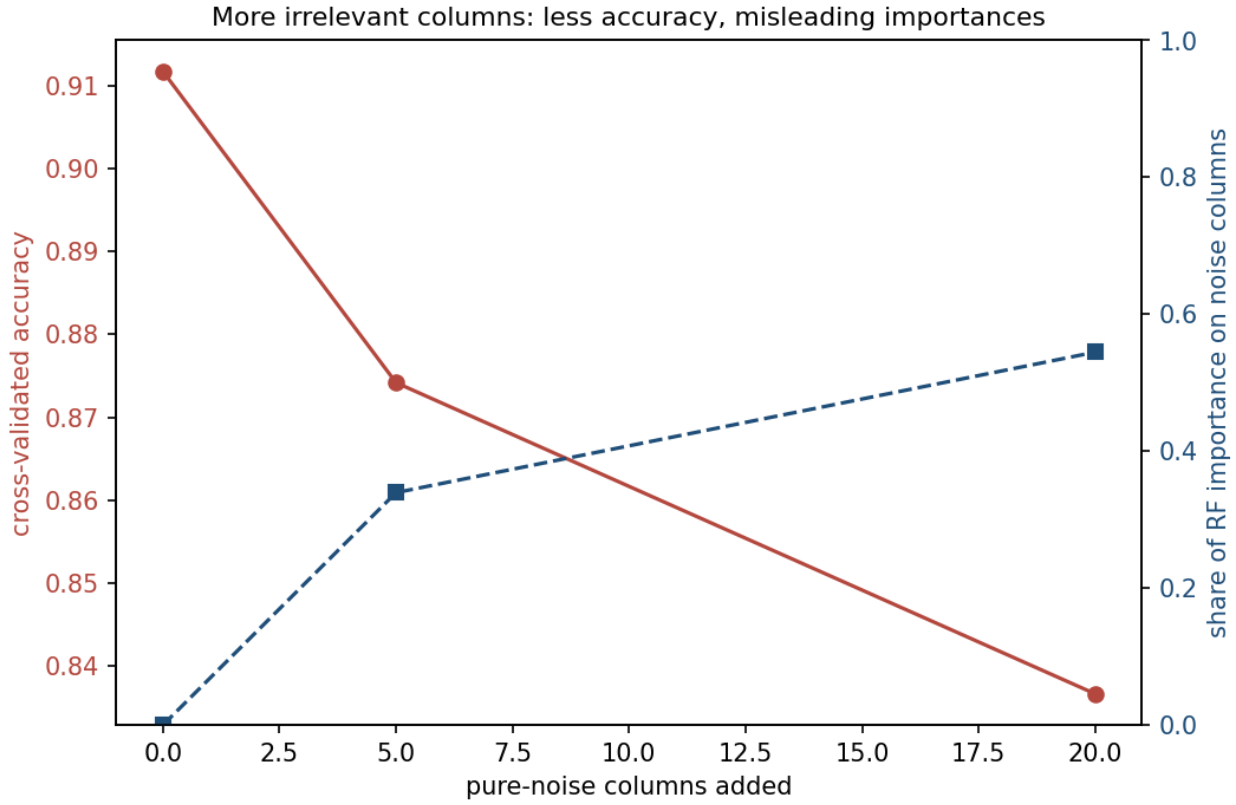
Test accuracy over 30 random 70/30 splits, single tree against bagging against random forest. Reading left to right: the median rises, the box narrows, and the whiskers pull in — mean and spread both improve together, exactly as sections 5.2 and 6 predicted from the $\rho\sigma^2$ floor.

7. Feature importance, and its limits

A random forest reports **feature importance**: the total impurity reduction each feature is responsible for, summed and averaged across every tree and every split. It is tempting to read this as "the forest tells you which features matter" — and with only the two real features in this dataset, it does exactly that.

The question is what happens once most of the columns carry nothing.

Noise columns added	Total columns	Cross-validated accuracy	Importance on noise columns
0	2	0.912	0.0%
5	7	0.874	33.9%
20	22	0.837	**54.5%**



Cross-validated accuracy (left axis) falls as pure-noise columns are added; the share of the forest's own importance that lands on those noise columns (right axis) rises at the same time. The two lines crossing is the whole warning: accuracy degrading gracefully gives no hint that the importance ranking behind it has become largely wrong.

More irrelevant columns cost accuracy — the same direction as lesson 6's curse of dimensionality, though the mechanism is different — and, more strikingly, an increasing share of the forest's own importance scores lands on columns that are `np.random.normal` noise by construction, wired to nothing. **With 20 pure-noise columns against 2 real features, 54% of the forest's importance mass sits on noise.**

The mechanism is quantifiable, from section 6's restriction directly. With `max_features = "sqrt"` and 22 total columns, each split considers $\lfloor\sqrt{22}\rfloor = 4$ candidates. By symmetry — the number of size-k subsets of p features that contain one fixed feature is $\binom{p-1}{k-1}$, against $\binom{p}{k}$ subsets in total — any one specific column, including either real feature, is a candidate with probability

$$\frac{\binom{p-1}{k-1}}{\binom{p}{k}} = \frac{k}{p} = \frac{4}{22} \approx 18.2\%$$

The middle equality is worth one line of cancellation, since it is what makes the answer so simple: writing both binomials as factorials,

$$\frac{(p-1)!}{(k-1)!\,(p-k)!} \cdot \frac{k!\,(p-k)!}{p!} = \frac{k!}{(k-1)!} \cdot \frac{(p-1)!}{p!} = \frac{k}{p}$$

— every factorial cancels except one factor at each end.

So any one column is *excluded* from a given split with probability $1 - 4/22 \approx 81.8\%$. For both real features to be left out together, all four candidates must come from the 20 noise columns, which happens with probability

$$\frac{\binom{20}{4}}{\binom{22}{4}} = \frac{4{,}845}{7{,}315} \approx 0.662$$

— about two splits in three offer no real feature at all, and the tree must choose its best option among whichever noise columns happened to be drawn — and among 1,200 finite, noisy rows, some noise column will show a nonzero ΔG purely by chance, every time. Each such gain is small, but it is not zero, and enough of them accumulate across hundreds of trees and thousands of splits to produce the shares measured above.

The predictable mistake, and why the instinct is sound. It is entirely reasonable to expect an average over hundreds of trees to wash out noise — and section 5.2's variance-reduction result is real, measured, and points the same way. The averaging genuinely reduces the *variance of the prediction*. It does not guarantee small importance for irrelevant columns, because the very restriction that decorrelates the trees and makes the forest work — forcing each split to choose from a random, incomplete menu — is the same restriction that occasionally hands a split to noise, with nothing better on the menu to give it instead.

The practical consequence: with few real features against many candidate columns, treat a random forest's importances as suggestive, not definitive — cross-check against **permutation importance** on held-out data (shuffle one column's values, breaking its link to the label, and measure how much accuracy drops: a column the model relies on costs accuracy, a noise column costs nothing), or against which columns could plausibly matter before asking the forest which ones do.

> **Try this:** in notebook 2, repeat the noise experiment with `max_features=None` (every feature considered at every split) instead of the default `"sqrt"`. The noise columns should receive less importance — because k/p rises to 1 and the exclusion probability that drives the effect falls to zero — at the cost of the variance reduction section 6 measured, since the trees are no longer forced to disagree.

---

8. Gradient boosting: correcting mistakes in sequence

Bagging and random forests build trees **independently** and average at the end. **Boosting** builds trees **in sequence**: each new tree is fit specifically to correct what the trees so far got wrong.

$$F_0(x) = \bar{y}, \qquad F_t(x) = F_{t-1}(x) + \alpha \cdot h_t(x)$$

where h_t is a shallow tree — depth 2 or 3, a **weak learner** — fit to what remains unexplained by F_{t-1}, and α is the **learning rate**: how much of each new tree's correction is actually applied.

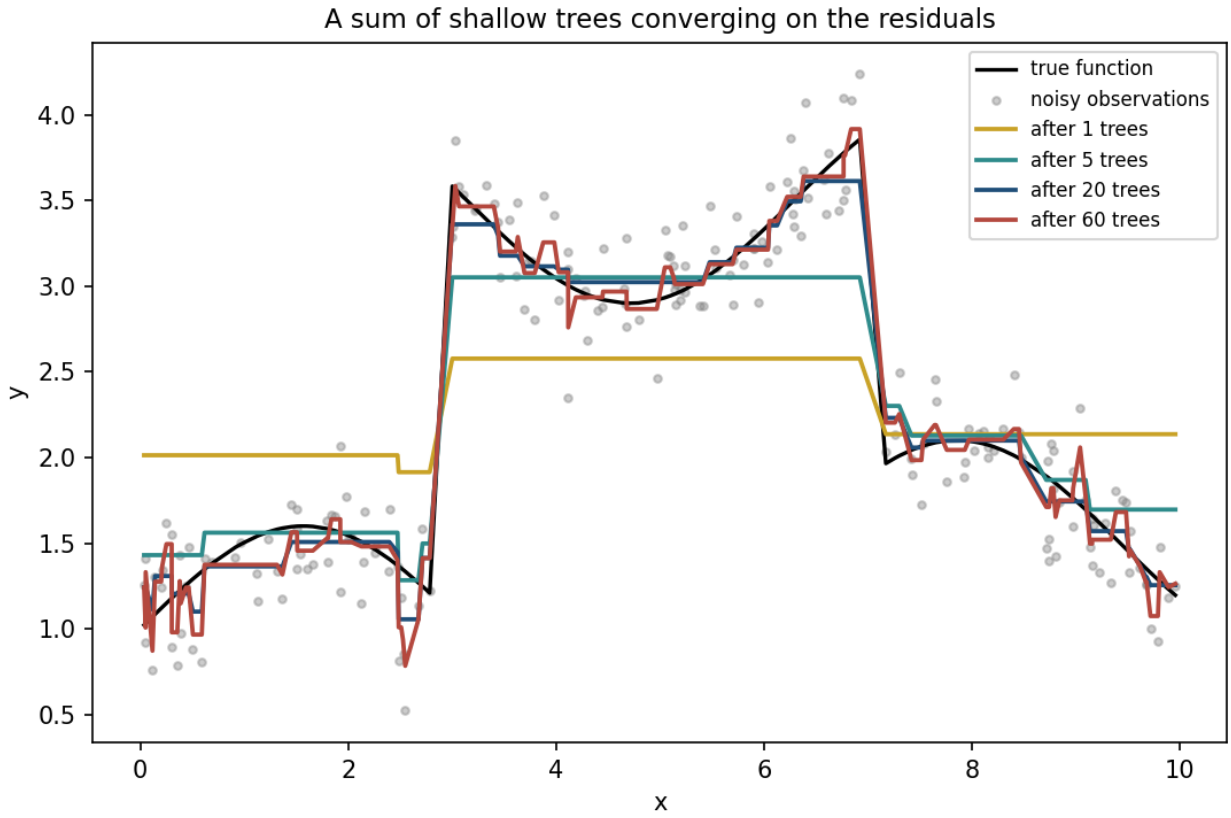
The trees are **regression trees**, even when the task is classification, because what they are fit to is a number, not a class. A regression tree is section 2's algorithm with two changes: each leaf predicts the mean of the values that reach it, and a split is scored by how much it lowers the squared error around those means, in place of the Gini impurity. The greedy search over features and thresholds is unchanged.

For squared-error regression, "what remains unexplained" has an exact name, the **residual** $y - F_{t-1}(x)$. This is a special case of a more general idea — **functional gradient descent**: h_t is fit to approximate the negative gradient of the loss with respect to F_{t-1} at each training point. For $L(y, F) = \frac{1}{2}(y - F)^2$,

$$-\frac{\partial L}{\partial F} = -(F - y) = y - F$$

so "fit the residual" and "fit the negative gradient" coincide exactly for squared error, and the general recipe reduces to the familiar one.

Worked example. A synthetic target — a three-level step function plus a gentle sine wave, observed with noise — was fit by 60 depth-2 trees at $\alpha = 0.3$:



A sum of shallow trees converging on the residuals. One tree barely moves the flat starting guess (mean squared error, MSE, against the true function: 0.395). Five trees sketch the coarse shape (MSE 0.068). Twenty trace the step function closely, oscillation included (MSE 0.012) — the ensemble's best point. By sixty trees the ensemble is tracking individual noisy observations as well as the underlying shape, and the error against the (noise-free) truth has risen back to 0.018: the same overfitting story as section 3's unconstrained tree, reached by a different route — not one tree grown too deep, but too many shallow trees each chasing whatever residual noise is left.

---

9. Boosting for classification: the same idea, a different gradient

Classification does not have a "residual" in the same literal sense, but the functional-gradient recipe still applies — only the loss changes. With $y \in \{0, 1\}$ and the ensemble's output converted to a probability by the logistic function, $p = \sigma(F) = 1/(1 + e^{-F})$, the loss is the **log-loss** lesson 4 used for logistic regression:

$$L(y, F) = -[y \log p + (1 - y) \log(1 - p)]$$

By the chain rule, $\dfrac{\partial L}{\partial p} = -\dfrac{y}{p} + \dfrac{1 - y}{1 - p}$ and $\dfrac{\partial p}{\partial F} = p(1 - p)$ (the logistic function's own derivative), so

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial p} \cdot \frac{\partial p}{\partial F} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p} \right) p(1 - p) = p - y$$

and the negative gradient — the **pseudo-residual** each tree is fit to — is $y - p$: the true label minus the current predicted probability, exactly the error term lesson 4's logistic regression gradient descent moved against. Boosting for classification is fitting a sequence of small trees to that same quantity, one correction at a time, rather than adjusting a fixed set of linear coefficients against it in one continuous descent.

Where it starts, and the first residuals. The starting score is the best constant, the log-odds of the base rate: 464 of the 1,200 applicants defaulted, $p = 0.387$, so $F_0 = \log(464/736) = -0.461$, which the logistic function turns back into 0.387 for every applicant. The first tree is then fit to $y - p$: **+0.613** for every applicant who defaulted and **−0.387** for every one who repaid. Where that tree finds a group of mostly defaulters, it raises their score; each later tree works on what the earlier ones left.

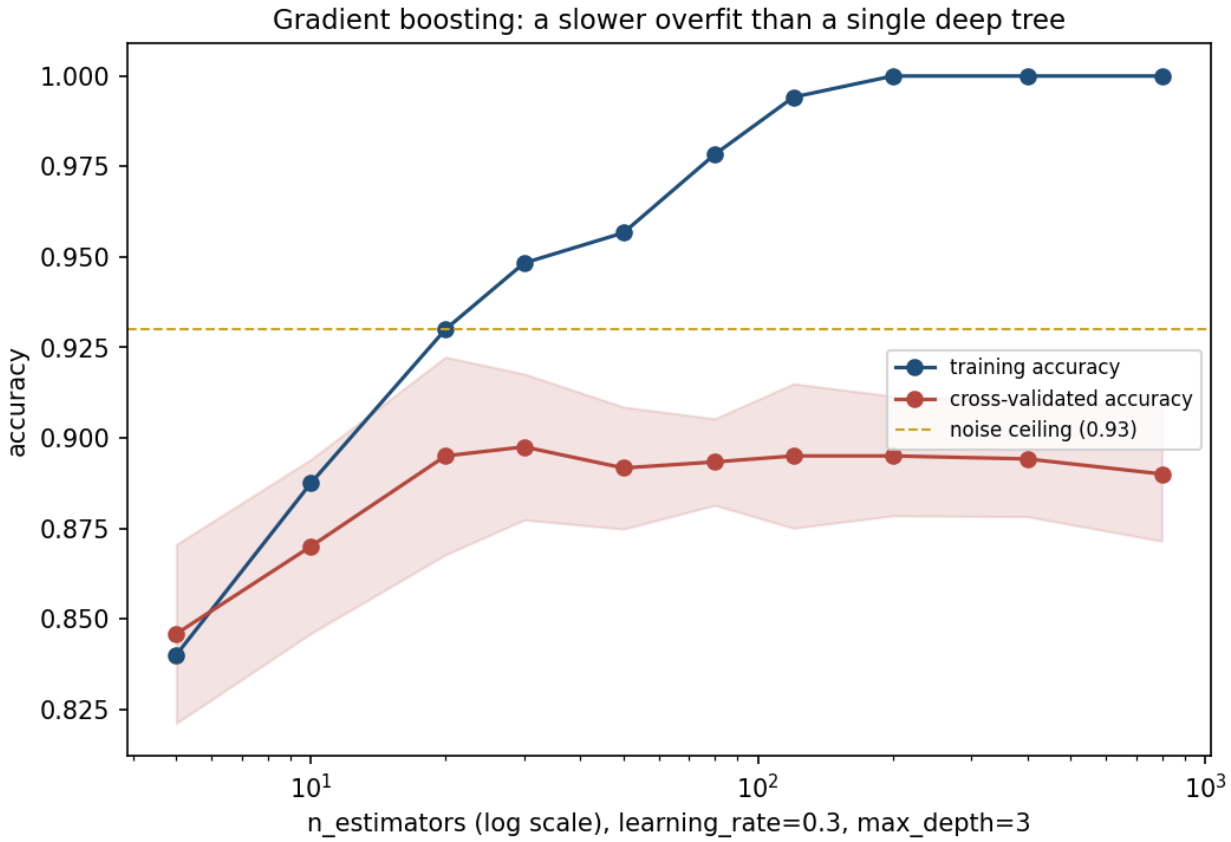
A default `GradientBoostingClassifier` — 100 trees, learning rate 0.1 — reaches **0.902 ± 0.020** cross-validated accuracy on the loan data, already ahead of any single tree.

---

10. Overfitting in boosting: two knobs that trade off

Unlike bagging, boosting keeps reducing training error for as long as it runs, because each tree is built specifically to reduce what remains. With nothing to stop it, that includes the 7% of labels that are noise by construction.

`n_estimators` ($\alpha = 0.3$)	Training accuracy	Cross-validated accuracy
5	0.840	0.846 ± 0.025
10	0.888	0.870 ± 0.024
20	0.930	0.895 ± 0.027
30	0.948	**0.897 ± 0.020**
80	0.978	0.893 ± 0.012
200	1.000	0.895 ± 0.017
800	1.000	0.890 ± 0.019



Training accuracy reaches 1.000 by 200 trees and stays there — the same ceiling section 3's unconstrained tree reached in one step, here approached gradually. Cross-validated accuracy climbs quickly, peaks around 30 trees, and drifts down by roughly a point over the following 770 trees.

The overfitting is real but **gentler** than a single unconstrained tree's: that tree lost 3 points of cross-validated accuracy past its peak (section 3); 800 boosted trees lose about 1, even though training accuracy has been at 1.000 since well before that point. That gentleness is why **early stopping** matters for boosting rather than being optional — the damage per added tree is small and easy to miss on a single run, and it is still there. Early stopping sets aside part of the training data, watches the score on it as trees are added, and stops once it has not improved for a fixed number of rounds; in scikit-learn it is the `n_iter_no_change` argument of `GradientBoostingClassifier`.

Learning rate and tree count are not independent: both control how much total correction the ensemble applies, so they trade off against each other. Fixing `n_estimators = 120` and varying only α:

Learning rate α	Training accuracy	Cross-validated accuracy
0.02	0.888	0.872 ± 0.025
0.05	0.914	0.897 ± 0.022
0.10	0.939	**0.902 ± 0.022**
0.30	0.994	0.895 ± 0.020
1.00	1.000	0.878 ± 0.020

A learning rate of 0.02 has not finished fitting the signal in 120 trees — training accuracy is still under 0.89. A learning rate of 1.0 has memorised the training set completely (1.000) and given back more than two points of cross-validated accuracy for it, 0.902 down to 0.878. The best setting sits in between: large enough to make real progress within the tree budget, small enough to leave each individual correction easy to outvote if it turns out to be wrong.

---

11. Choosing among trees, forests and boosting

Every method from this lesson, on the same five folds:

Model	Cross-validated accuracy	Fraction of the 0.93 ceiling
Majority baseline	0.613	0.659
Logistic regression	0.748	0.805
Single tree, unconstrained	0.852	0.916
Single tree, `max_depth = 8`	0.882	0.949
Gradient boosting, 30 trees	0.897	0.965
Bagging, 100 trees	0.904	0.972
Random forest, 100 trees	**0.911**	**0.979**

Noise ceiling: 0.93 (1 − the 7% deliberately flipped labels). No method reaches it, and none should — a model that classified the flipped labels "correctly" would be modelling the noise, not the rule.

Three findings, read together, are this lesson's real content. First, every ensemble — bagging, the random forest, gradient boosting — lands within about a point and a half of the others and closer to the ceiling than either single tree, tuned or not: which ensembling *strategy* you pick matters far less than whether you ensemble at all. Second, the single tuned tree is not far behind them, at 0.882 against the random forest's 0.911, and it is the only model on this table a person could read start to finish — the accuracy-interpretability trade this lesson opened with, restated as numbers. Third, the unconstrained tree is the worst model on this list despite being, in a narrow sense, the most flexible one: flexibility without a stopping rule is not an advantage, a point this course has now made with several different methods.

Situation	Reach for
The decision must be explained to the person it affects	A single tree, small enough to read; if even that is too opaque, none of these models
Tabular data, accuracy matters most, no explanation required	A random forest: the best score here, at its default settings
Accuracy matters, and there is time to tune the number of trees and the learning rate together	Gradient boosting, tuned on validation data
Many columns, few of them informative	Any tree method, checking its importances by permutation: section 7's 54% came with 1,200 rows
Too few rows to set a validation set aside	A random forest, judged by its out-of-bag score

One caveat, in lesson 5's terms. Depth 8 and 30 boosted trees were chosen because they scored best on the same five folds as the leaderboard, so those two leaderboard rows are slightly optimistic;

nested cross-validation would remove the bias. Bagging and the random forest ran at their defaults, chose nothing on these folds, and are not affected — and they are still the two best rows.

Summary

- A decision tree splits greedily on Gini impurity, $G = 1 - \sum_c p_c^2$, one feature and threshold at a time — verified here against scikit-learn's implementation, prediction by prediction.
- Depth is the bias-variance dial. Cross-validated accuracy peaked at **depth 8** (0.882); an unconstrained tree reached training accuracy **1.000** and lost 3 points of cross-validated accuracy for it.
- A tree's strongest split survives resampling almost unchanged; its weaker splits do not, and disagreement between two 65%-resampled trees reached 11.7% of predictions.
- Bagging averages trees fit to bootstrap resamples. Each sample leaves about **36.8%** of rows out-of-bag — the standard limit $(1 - 1/m)^m \to e^{-1}$ — which is why the OOB score is free validation; across twelve seeds it tracked 5-fold cross-validation to within 0.003.
- Averaged variance falls towards a floor of $\rho\sigma^2$, not zero. A random forest lowers ρ by restricting each split to a random subset of features, tightening test-accuracy spread from ± 0.019 (single tree) to ± 0.015 (bagging) to ± 0.014 (random forest) over 30 resampled splits.
- **With 20 pure-noise columns against 2 real features, 54% of a random forest's feature importance landed on noise** — the number worth carrying out of this lesson. The mechanism is exact: `max_features="sqrt"` excludes any one real feature from 81.8% of splits.
- Gradient boosting fits shallow trees in sequence to the negative gradient of the loss — the residual for squared error, $y - p$ for log-loss, the same quantity lesson 4's logistic regression descended against. Cross-validated accuracy peaked near 30 trees and drifted down by about a point over the next 770, a gentler overfit than a single unconstrained tree's but not an absent one.
- On this dataset, every ensemble method landed within about a point and a half of the others, all closer to the noise ceiling than any single tree.

Homework

`Exercises/07_trees_and_ensembles.md`, discussed at the start of Lesson 8, **Friday 20 November 2026**.

Notation used in this lesson

Symbol	Meaning
G	Gini impurity of a group of examples
ΔG	impurity reduction a candidate split buys
p_c	fraction of a group belonging to class c
d	a tree's depth
m, n	number of examples, number of features
m_g; m_P, m_L, m_R	examples in a group; in a parent and its two children
σ^2, ρ	variance of one tree's prediction, correlation between two trees

Symbol	Meaning
B	number of trees averaged
F_t, h_t	the ensemble's prediction after t rounds, and the t-th tree added to it
α	**learning rate**, boosting's shrinkage factor — the same symbol as lessons 3 and 9
p	the model's predicted probability, $\sigma(F)$

Further reading

Resource	Type	Why read it
scikit-learn: decision trees	Official docs	The splitting criteria and stopping parameters stated precisely
scikit-learn: ensemble methods	Official docs	Bagging, random forests and boosting side by side, with the parameters that matter
Breiman, *Random Forests* (2001)	Paper	The original derivation of the $\rho\sigma^2$ variance bound used in section 5.2
Friedman, *Greedy Function Approximation: A Gradient Boosting Machine* (2001)	Paper	The functional-gradient view of boosting, section 8's general recipe, from its source
Hastie, Tibshirani & Friedman, *Elements of Statistical Learning*, ch. 9–10, 15	Book	Trees (ch. 9), boosting (ch. 10) and random forests (ch. 15), the definitive treatment
Molnar, *Interpretable Machine Learning*, ch. 5–6	Book, free online	Permutation importance and its failure modes — the fix section 7 points towards